Transaction Witness

개요

Transaction Witness는 개별 트랜잭션의 실행과 관련된 모든 데이터를 담고 있는 구조체입니다. `Transaction` 구조체는 트랜잭션의 메타데이터, 서명 정보, 실행 단계, 호출 정보 등을 포함하며, Tx Circuit, Sig Circuit, RLP Circuit 등 여러 회로에서 사용됩니다.

Transaction 구조체

정의

`Transaction` 구조체는 `zkevm-circuits/src/witness/tx.rs` 에 정의되어 있으며, 다음과 같은 주요 필드를 포함합니다:

```
pub struct Transaction {
/// 이 트랜잭션이 포함된 블록 번호
pub block_number: u64,
/// 블록 내 트랜잭션 식별자
pub id: usize,
/// 트랜잭션 해시
pub hash: H256,
/// 트랜잭션 타입
pub tx_type: TxType,
/// 발신자 계정의 nonce
```

```
pub nonce: u64,
/// 트랜잭션의 가스 한도
pub gas: u64,
/// 가스 가격
pub gas_price: Word,
/// 최대 가스 수수료 (EIP1559)
pub max_fee_per_gas: Word,
/// 최대 우선순위 수수료 (EIP1559)
pub max_priority_fee_per_gas: Word,
/// 발신자 주소
pub caller_address: Address,
/// 수신자 주소
pub callee_address: Option<Address>,
/// CREATE 트랜잭션 여부
pub is_create: bool,
/// 전송 이더 금액
pub value: Word,
/// 호출 데이터
pub call_data: Vec<u8>,
/// 호출 데이터 길이
pub call_data_length: usize,
/// 호출 데이터의 가스 비용
pub call_data_gas_cost: u64,
```

```
/// 액세스 리스트 가스 비용 (EIP 2930)
pub access_list_gas_cost: u64,
/// 서명되지 않은 트랜잭션의 RLP 인코딩 바이트의 가스 비용
pub tx_data_gas_cost: u64,
/// 체인 ID (EIP-155)
pub chain_id: u64,
/// 서명되지 않은 트랜잭션의 RLP 인코딩 바이트
pub rlp_unsigned: Vec<u8>,
/// 서명된 트랜잭션의 RLP 인코딩 바이트
pub rlp_signed: Vec<u8>,
/// 트랜잭션 서명의 "v" 값
pub v: u64,
/// 트랜잭션 서명의 "r" 값
pub r: Word,
/// 트랜잭션 서명의 "s" 값
pub s: Word,
/// 현재 L1 수수료 값
pub l1_fee: TxL1Fee,
/// 커밋된 L1 수수료 값
pub l1_fee_committed: TxL1Fee,
/// EIP-2930 액세스 리스트 (선택적)
pub access_list: Option<AccessList>,
/// 트랜잭션에서 수행된 호출들
```

```
pub calls: Vec<Call>,
/// 트랜잭션에서 실행된 단계들
pub steps: Vec<ExecStep>,
}
```

트랜잭션 타입

```
pub enum TxType {
PreEip155, // EIP-155 이전 트랜잭션
Eip155, // EIP-155 트랜잭션
Eip2930, // EIP-2930 (액세스 리스트)
Eip1559, // EIP-1559 (동적 수수료)
L1Msg, // L1에서 L2로의 메시지
}
```

트랜잭션 데이터 변환

tx_convert 함수

`tx_convert` 함수는 Bus-Mapping의 트랜잭션 데이터를 witness 형식으로 변환합니다:

```
pub(super) fn tx_convert(
tx: &circuit_input_builder::Transaction,
id: usize,
```

```
chain_id: u64,
next_block_num: u64,
) -> Transaction
```

변환 과정

1. **기본 정보 추출**

```
let  callee_address = tx.to;
let  tx_gas_cost = if  tx.tx_type.is_l1_msg() {
0
} else {
tx_data_gas_cost(&tx.rlp_bytes)
};
```

- 수신자 주소 설정
- L1Msg가 아닌 경우 RLP 바이트의 가스 비용 계산

2. **가스 수수료 처리**

```
max_fee_per_gas: if  tx.tx_type.is_eip1559() {
tx.gas_fee_cap
} else {
tx.gas_price
},
max_priority_fee_per_gas: if  tx.tx_type.is_eip1559() {
```

```
tx.gas_tip_cap

} else {

tx.gas_price

},
```

- EIP-1559 트랜잭션의 경우 별도의 수수료 필드 사용
- 그 외의 경우 gas_price 사용

3. **호출 정보 변환**

```
calls: tx

.calls()

.iter()

.map(|call| Call {

id: call.call_id,

is_root: call.is_root,

is_create: call.is_create(),

code_hash: call.code_hash.to_word(),

// ... 기타 필드

})

.collect(),
```

- 각 호출을 witness 형식으로 변환

4. **실행 단계 변환**

```
steps: tx
```

```
.steps()
.iter()
.map(|step| step_convert(step, tx.block_num))
.chain({
// EndInnerBlock 단계 추가
(tx.block_num..next_block_num)
.map(|block_num| ExecStep {
execution_state: ExecutionState::EndInnerBlock,
block_num,
// ...
})
.collect::<Vec<ExecStep>>()
})
.collect(),
```

- 각 실행 단계를 witness 형식으로 변환
- 블록 경계를 넘는 경우 EndInnerBlock 단계 추가

서명 데이터 생성

sign_data 메서드

```
pub fn sign_data(&self) -> Result<SignData<secp256k1::Fq, secp256k1::Secp256k1
```

트랜잭션 서명으로부터 Sig Circuit에 필요한 데이터를 생성합니다:

1. **서명 값 추출**

```
let  sig_r_le = self.r.to_le_bytes();
let  sig_s_le = self.s.to_le_bytes();
let  sig_r = ct_option_ok_or(secp256k1::Fq::from_repr(sig_r_le), Error::Signatur
let  sig_s = ct_option_ok_or(secp256k1::Fq::from_repr(sig_s_le), Error::Signatur
```

2. **메시지 해시 계산**

```
let  msg = self.rlp_unsigned.clone().into();
let  msg_hash = keccak256(&self.rlp_unsigned);
```

3. **공개키 복구**

```
let  v = self.tx_type.get_recovery_id(self.v);
let  pk = recover_pk2(v, &self.r, &self.s, &msg_hash)?;
```

4. **메시지 해시 모듈로 연산**

```
let  msg_hash = BigUint::from_bytes_be(msg_hash.as_slice());
let  msg_hash = msg_hash.mod_floor(&*SECP256K1_Q);
let  msg_hash_le = biguint_to_32bytes_le(msg_hash);
let  msg_hash = ct_option_ok_or(secp256k1::Fq::from_repr(msg_hash_le), Error::Si
```

5. **SignData 구조체 반환**

```
Ok(SignData {
signature: (sig_r, sig_s, v),
pk,
msg,
msg_hash,
})
```

Tx Table 할당

table_assignments_fixed 메서드

```
pub fn table_assignments_fixed<F: Field>(
&self,
challenges: Challenges<Value<F>>,
) -> Vec<[Value<F>; 5]>
```

Tx Table에 고정 필드를 할당합니다:

- **Nonce**: 발신자 nonce
- **GasPrice**: 가스 가격 (RLC 인코딩)
- **Gas**: 가스 한도
- **CallerAddress**: 발신자 주소
- **CalleeAddress**: 수신자 주소

- **IsCreate**: CREATE 트랜잭션 여부
- **Value**: 전송 금액 (RLC 인코딩)
- **CallDataRLC**: 호출 데이터 RLC
- **CallDataLength**: 호출 데이터 길이
- **CallDataGasCost**: 호출 데이터 가스 비용
- **TxDataGasCost**: 트랜잭션 데이터 가스 비용
- **ChainID**: 체인 ID
- **SigV, SigR, SigS**: 서명 값
- **TxSignLength, TxSignRLC, TxSignHash**: 서명 데이터
- **TxHashLength, TxHashRLC, TxHash**: 트랜잭션 해시 데이터
- **TxType**: 트랜잭션 타입
- **AccessListAddressesLen, AccessListStorageKeysLen**: 액세스 리스트 크기
- **AccessListRLC**: 액세스 리스트 RLC
- **MaxFeePerGas, MaxPriorityFeePerGas**: EIP-1559 수수료
- **BlockNumber**: 블록 번호

table_assignments_dyn 메서드

```
pub fn table_assignments_dyn<F: Field>(
&self,
_challenges: Challenges<Value<F>>,
) -> Vec<[Value<F>; 5]>
```

호출 데이터를 바이트 단위로 할당합니다:

```
self.call_data
.iter()
.enumerate()
.map(|(idx, byte)| {
[
Value::known(F::from(self.id as u64)),
Value::known(F::from(TxContextFieldTag::CallData as u64)),
Value::known(F::from(idx  as u64)),
Value::known(F::from(*byte  as u64)),
Value::known(F::zero()),
]
})
.collect()
```

table_assignments_access_list_dyn 메서드

```
pub  fn  table_assignments_access_list_dyn<F: Field>(
&self,
challenges: Challenges<Value<F>>,
) -> Vec<[Value<F>; 5]>
```

액세스 리스트를 할당합니다:

1. **주소 할당**

```
assignments.push([
Value::known(F::from(self.id as u64)),
Value::known(F::from(TxContextFieldTag::AccessListAddress as u64)),
Value::known(F::from((al_idx + 1) as u64)),
Value::known(al.address.to_scalar().unwrap()),
Value::known(al.address.to_scalar().unwrap()),
]);
```

2. **스토리지 키 할당**

```
for  sk  in  al.storage_keys.iter() {
sks_acc += 1;
assignments.push([
Value::known(F::from(self.id as u64)),
Value::known(F::from(TxContextFieldTag::AccessListStorageKey as u64)),
Value::known(F::from(sks_acc  as u64)),
rlc_be_bytes(&sk.to_fixed_bytes(), challenges.evm_word()),
Value::known(al.address.to_scalar().unwrap()),
]);
}
```

RLP Witness 생성

gen_rlp_witness 메서드

```
pub(crate) fn gen_rlp_witness<F: Field>(
&self,
is_hash: bool,
challenges: &Challenges<Value<F>>,
) -> Vec<RlpFsmWitnessRow<F>>
```

RLP Circuit을 위한 witness를 생성합니다:

1. RLP 바이트 및 포맷 선택

```
let (rlp_bytes, format) = if is_hash {
(
self.rlp_signed.clone(),
match self.tx_type {
TxType::Eip155 => TxHashEip155,
TxType::PreEip155 => TxHashPreEip155,
TxType::Eip1559 => TxHashEip1559,
TxType::L1Msg => L1MsgHash,
TxType::Eip2930 => TxHashEip2930,
},
)
} else {
(
```

```
self.rlp_unsigned.clone(),

match  self.tx_type {

TxType::Eip155 => TxSignEip155,

TxType::PreEip155 => TxSignPreEip155,

TxType::Eip1559 => TxSignEip1559,

TxType::Eip2930 => TxSignEip2930,

_ => unreachable!("tx type {:?} not supported", self.tx_type),

},

)

};
```

2. RLP 디코딩 상태 머신

RLP FSM(Finite State Machine)을 사용하여 RLP 인코딩을 디코딩합니다:

상태 종류:

- **DecodeTagStart**: 태그 디코딩 시작
- **Bytes**: 바이트 시퀀스 읽기
- **LongBytes**: 긴 바이트 시퀀스 (길이 > 55)
- **LongList**: 긴 리스트 (길이 > 55)

RLP 인코딩 규칙:

- `0x00-0x7f` : 단일 바이트
- `0x80` : 빈 문자열
- `0x81-0xb7` : 짧은 문자열 (길이 1-55)
- `0xb8-0xbf` : 긴 문자열 (길이 > 55)

- `0xc0-0xf7` : 짧은 리스트
- `0xf8-0xff` : 긴 리스트

3. 스택 연산 추적

```
let mut stack_ops: Vec<RlpStackOp<F>> = vec![];
```

RLP 디코딩 과정에서 발생하는 스택 연산을 추적:

- **PUSH**: 새로운 깊이 레벨 진입
- **POP**: 깊이 레벨 종료
- **UPDATE**: 같은 깊이에서 바이트 읽기

4. 액세스 리스트 추적

```
let mut access_list_idx: u64 = 0;
let mut storage_key_idx: u64 = 0;
```

액세스 리스트의 주소와 스토리지 키에 고유 식별자 할당

5. Witness 행 생성

각 RLP 바이트에 대해 witness 행을 생성:

```
witness.push(RlpFsmWitnessRow {
rlp_table: RlpTable {
tx_id,
format,
rlp_tag,
```

```
tag_value,
tag_bytes_rlc,
tag_length,
is_output,
is_none,
access_list_idx,
storage_key_idx,
},
state_machine: StateMachine {
state: cur.state,
tag: cur.tag,
byte_idx: cur.byte_idx + 1,
byte_value,
// ... 기타 필드
},
rlp_decoding_table: RlpStackOp::default(),
});
```

6. 스택 연산 정렬 및 할당

```
stack_ops.sort_by(|a, b| {
if (a.tx_id, a.format as u64, a.depth, a.al_idx, a.sk_idx, a.byte_idx, a.stack_o
> (b.tx_id, b.format as u64, b.depth, a.al_idx, a.sk_idx, b.byte_idx, b.stack_op
```

```
{
std::cmp::Ordering::Greater
} else {
std::cmp::Ordering::Less
}
});


for (idx, op) in  stack_ops.into_iter().enumerate() {
witness[idx].rlp_decoding_table = op;
}
```

RlpFsmWitnessGen 트레이트 구현

gen_sm_witness 메서드

```
fn  gen_sm_witness(&self, challenges: &Challenges<Value<F>>) -> Vec<RlpFsmWitnes
let  hash_wit = self.gen_rlp_witness(true, challenges);
let  sign_wit = match  self.tx_type {
TxType::L1Msg => vec![],
_ => self.gen_rlp_witness(false, challenges),
};
[sign_wit, hash_wit].concat()
```

```
}
```

- 서명 witness와 해시 witness를 모두 생성
- L1Msg 타입은 서명이 없으므로 해시 witness만 생성

gen_data_table 메서드

```
fn gen_data_table(&self, challenges: &Challenges<Value<F>>) -> Vec<DataTable<F>
```

RLP 바이트를 DataTable 형식으로 변환:

```
let get_table = |rlp_bytes: &Vec<u8>, format: Format| {
let n = rlp_bytes.len();
rlp_bytes
.iter()
.enumerate()
.scan(
(Value::known(F::zero()), Value::known(F::zero())),
|(rlc, gas_cost_acc), (i, &byte_value)| {
let byte_cost = if byte_value == 0 { 4 } else { 16 };
*rlc = *rlc * r + Value::known(F::from(byte_value as u64));
*gas_cost_acc = *gas_cost_acc + Value::known(F::from(byte_cost));
Some(DataTable {
tx_id,
format,
```

```
byte_idx: i + 1,
byte_rev_idx: n - i,
byte_value,
bytes_rlc: *rlc,
gas_cost_acc: *gas_cost_acc,
})
},
)
.collect::<Vec<_>>()
};
```

- 각 바이트에 대해 RLC와 누적 가스 비용 계산
- 0 바이트는 4 가스, 비-0 바이트는 16 가스

더미 트랜잭션

dummy 메서드

```
pub fn dummy(chain_id: u64) -> Self
```

테스트 및 패딩용 더미 트랜잭션 생성:

```
let (dummy_tx, dummy_sig) = get_dummy_tx();
let dummy_tx_hash = get_dummy_tx_hash();
let rlp_signed = dummy_tx.rlp_signed(&dummy_sig).to_vec();
```

```
let  rlp_unsigned = dummy_tx.rlp_unsigned().to_vec();



Self {

block_number: 0,

id: 0,

caller_address: Address::zero(),

callee_address: Some(Address::zero()),

is_create: false,

chain_id,

tx_data_gas_cost: tx_data_gas_cost(&rlp_signed),

v: dummy_sig.v,

r: dummy_sig.r,

s: dummy_sig.s,

rlp_signed,

rlp_unsigned,

hash: dummy_tx_hash,

tx_type: PreEip155,

..Default::default()

}
```

유틸리티 메서드

is_chunk_l2_tx 메서드

```
pub fn is_chunk_l2_tx(&self) -> bool {
self.tx_type != TxType::L1Msg && !self.caller_address.is_zero()
}
```

- 청크의 txbytes에 포함되는 트랜잭션인지 확인
- L1Msg가 아니고 패딩이 아닌 경우 true

요약

Transaction Witness는 개별 트랜잭션의 모든 실행 데이터를 담고 있습니다:

1. **메타데이터**: 트랜잭션 타입, nonce, 가스, 주소 등
2. **서명 정보**: v, r, s 값과 RLP 인코딩 바이트
3. **실행 데이터**: 호출 정보와 실행 단계
4. **테이블 할당**: Tx Table, RLP Table, DataTable 생성
5. **RLP 처리**: FSM 기반 RLP 디코딩 witness 생성
6. **서명 검증**: Sig Circuit을 위한 SignData 생성

Transaction Witness는 `tx_convert` 함수를 통해 Bus-Mapping의 출력으로부터 생성되며, 여러 회로에서 트랜잭션 관련 제약 조건을 검증하는 데 사용됩니다.